МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ

по лабораторной работе №3
по дисциплине «Построение и анализ алгоритмов»
Вариант 14б

Студент гр. 4343 _____ Едина А.

Преподаватель _____ Жангиров Т. Р.

Санкт-Петербург
2026

Задание

1 задание.

Над строкой $\varepsilon\varepsilon$ (будем считать строкой непрерывную последовательность из латинских букв) заданы следующие операции:

1. *replace(εε, a, b)* – заменить символ *a* на символ *b*.
2. *insert(εε, a)* – вставить в строку символ *a* (на любую позицию).
3. *delete(εε, b)* – удалить из строки символ *b*.

Каждая операция может иметь некоторую цену выполнения (*положительное число*).

Даны две строки A*A* и B*B*, а также три числа, отвечающие за цену каждой операции. Определите минимальную стоимость операций, которые необходимы для превращения строки A*A* в строку B*B*.

Входные данные: первая строка – три числа: цена операции *replace*, цена операции *insert*, цена операции *delete*; вторая строка – A*A*; третья строка – B*B*.

Выходные данные: одно число – минимальная стоимость операций.

2 задание.

Над строкой $\varepsilon\varepsilon$ (будем считать строкой непрерывную последовательность из латинских букв) заданы следующие операции:

1. *replace(εε, a, b)* – заменить символ *a* на символ *b*.
2. *insert(εε, a)* – вставить в строку символ *a* (на любую позицию).
3. *delete(εε, b)* – удалить из строки символ *b*.

Каждая операция может иметь некоторую цену выполнения (*положительное число*).

Даны две строки A*A* и B*B*, а также три числа, отвечающие за цену каждой операции. Определите последовательность операций (редакционное предписание) с минимальной стоимостью, которые необходимы для превращения строки A*A* в строку B*B*.

Пример (все операции стоят одинаково)

M	**M**	**M**	**R**	**I**	**M**	**R**	**R**
C	**O**	**N**	**N**		**E**	**C**	**T**
C	O	N	**E**	**H**	E	A	D

Пример (цена замены 3, остальные операции по 1)

M	**M**	**M**	**D**	**M**	**I**	**I**	**I**	**I**	**D**	**D**
C	**O**	**N**	**N**	**E**					**C**	**T**
C	O	N		E	**H**	**E**	**A**	**D**		

Входные данные: первая строка – три числа: цена операции *replace*, цена операции *insert*, цена операции *delete*; вторая строка – A*A*; третья строка – B*B*.

Выходные данные: первая строка – последовательность операций (M – совпадение, ничего делать не надо; R – заменить символ на другой; I – вставить символ на текущую позицию; D – удалить символ из строки); вторая строка – исходная строка A; третья строка – исходная строка B.

3 задание.

Расстоянием Левенштейна назовём минимальное количество операций вставки одного символа, удаления одного символа и замены одного символа на другой, необходимых для превращения одной строки в другую. Разработайте программу, осуществляющую поиск расстояния Левенштейна между двумя строками.

Пример:

Для строк pedestal и stien расстояние Левенштейна равно 7:

- Сначала нужно совершить четыре операции удаления символа: pedestal -> stal.
- Затем необходимо заменить два последних символа: stal -> stie.
- Потом нужно добавить символ в конец строки: stie -> stien.

Параметры входных данных:

Первая строка входных данных содержит строку из строчных латинских букв. (SS, 1≤|S|≤25501≤|S|≤2550). Вторая строка входных данных содержит строку из строчных латинских букв. (TT, 1≤|T|≤25501≤|T|≤2550).

Параметры выходных данных:

Одно число LL, равное расстоянию Левенштейна между строками SS и TT.

Индивидуальное задание.

Вар. 14. Два подварианта. Методом динамического программирования вычислить:

14б. длину наибольшей общей подстроки двух строк, вывести и саму эту подстроку.

Выполнение работы.

1 задание.

Файл leven_equal.py

Программа считывает три целых числа – стоимости операций replace, insert, delete – и две строки first и second. Строится матрица table размером (first_len+1) × (second_len+1), где table[i][j] – минимальная стоимость преобразования первых i символов строки first в первые j символов строки second.

Основной цикл по всем i от 1 до first_len и j от 1 до second_len. Для каждой пары символов вычисляется стоимость m = 0, если символы совпадают, иначе m = rep. Затем table[i][j] = min(table[i-1][j] + dele, table[i][j-1] + ins, table[i-1][j-1] + m).

Ответом является значение table[first_len][second_len].

2 задание.

Файл edit_script.py

После заполнения матрицы table (аналогично заданию 1) выполняется обратный проход от (first_len, second_len) до (0,0) для восстановления последовательности операций.

Переменная prescripton собирает символы операций в обратном порядке, поэтому в конце строка переворачивается.

Логика восстановления (в цикле while i > 0 or j > 0):

1. Если i > 0 и j > 0 и выполняется равенство table[i][j] == table[i-1][j-1] + (0 if first[i-1]==second[j-1] else rep), то добавляется:
   - "M", если символы одинаковы,
   - "R", если требуется замена.
   - После этого i -= 1, j -= 1.
2. Иначе, если i > 0 и table[i][j] == table[i-1][j] + dele, добавляется "D" (удаление) и i -= 1.

3. Иначе, если j > 0 и table[i][j] == table[i][j-1] + ins, добавляется "I" (вставка) и j -= 1.

После цикла полученная строка prescripton выводится в обратном порядке ([::-1]). Затем выводятся исходные строки first и second.

3 задание.

Файл levenstein.py

Для экономии памяти используется одномерный метод динамического программирования. Хранятся два списка: prev (предыдущая строка) и curr (текущая).

- Инициализация: prev = list(range(m + 1)), где m = len(second). Это соответствует dp[0][j] = j.
- Основной цикл для i от 1 до n (n = len(first)):
  - curr = [i] + [0] * m – первый элемент curr[0] = i(стоимость удаления всех символов first).
  - Для j от 1 до m:
    - cost = 0 if first[i-1] == second[j-1] else 1
    - curr[j] = min(prev[j] + 1, curr[j-1] + 1, prev[j-1] + cost)
  - После завершения внутреннего цикла prev = curr.
- Ответ: prev[m].

Индивидуальное задание.

Файл lb3.py

Реализована функция LCS(str1, str2), которая возвращает наибольшую общую подстроку и её длину.

Используется двумерная матрица lcs, где lcs[i][j] – длина наибольшей общей подстроки, оканчивающейся на позиции i-1 в str1 и j-1 в str2.

- Инициализация: все элементы lcs равны 0.
- Заполнение для i от 1 до len(str1) и j от 1 до len(str2)
- После окончания циклов, если maxlen > 0, подстрока извлекается как str1[endpos - maxlen : endpos]. Иначе возвращается сообщение об отсутствие общих подстрок и длина 0.

•

Исследование

Задание 1

Пространственная сложность

Хранится вся матрица расстояний. Количество ячеек:

(first_len + 1) * (second_len + 1).

Обозначим n = len(first), m = len(second).

Тогда сложность по памяти: O(n * m).

Временная сложность

Для заполнения каждой из n * m ячеек выполняется вычисление минимума из трёх значений (константное число операций). Инициализация первого столбца и первой строки требует O(n + m).

Таким образом, общая временная сложность: O(n * m).

Задание 2

Пространственная сложность

Алгоритм сохраняет полную матрицу table размером (n+1)*(m+1) для последующего восстановления последовательности операций. Дополнительно хранится строка prescripton длиной не более n + m.

Итого: O(n * m).

Временная сложность

Заполнение матрицы: O(n * m).

Обратный проход: в худшем случае количество шагов равно n + m (каждый шаг уменьшает i или j), на каждом шаге выполняется константное количество проверок.

Итого: O(n * m).

Задание 3

Пространственная сложность

Вместо полной матрицы хранятся два массива длины m+1 (prev и curr). Дополнительно используются скалярные переменные.

Итого: O(m), где m = len(second).

Временная сложность

Алгоритм последовательно обрабатывает все n строк и для каждой из них вычисляет m значений. На каждую ячейку затрачивается O(1) операций (вычисление минимума из трёх слагаемых).

Итого: O(n * m).

Индивидуальное задание

Пространственная сложность

Необходимо хранить полную матрицу dp размером (n+1)*(m+1), где n = len(str1), m = len(str2). Дополнительно – две скалярные переменные maxlen и endpos.

Итого: O(n * m).

Временная сложность

Заполнение каждой из n * m ячеек требует выполнения сравнения символов и пары присваиваний – O(1) операций. Обход матрицы совершается за два вложенных цикла.

Итого: O(n * m).

Тестирование

Таблица 1 - Результаты тестирование для задания 1

Тест	Входные данные	Выходные данные
1	1 1 1 abc abc	0
2	2 1 1 ab ba	2
3	1 1 1 kitten sitting	3
4	10 1 1 abc def	6
5	1 10 1 hello healooo	21

Таблица 2 - Результаты тестирования для задания 2

Тест	Входные данные	Выходные данные
1	1 1 1 hahaha hahaha	MMMMMM hahaha hahaha
2	1 1 1 abc	III abc
3	1 1 1 cat dpg	RRR cat dpg
4	10 1 1 here hire	MIDMM here hire
5	1 1 1 someinterestingtext wowwhereiam	RMDDRRRMMMDDMDDDDRR someinterestingtext wowwhereiam

Таблица 3 - Результаты тестирования для задания 3

Тест	Входные данные	Выходные данные
1	qwerty qwerty	0
2	kitten sitting	3
3	qwe zxc	3
4	hi hellomyfriends	12

5	zxcvbn nbvcxz	6

Таблица 4 - Результаты тестирования для индивидуального задания

Тест	Входные данные	Выходные данные
1	abcdef abcxyz	('abc', 3)
2	xxhello world yy zzhello world tt	('hello world ', 12)
3	abcdabcef abcefabcd	('abcef', 5)
4	qwerty asdfgh	('Общих подстрок не существует', 0)
5	meow meowmeow	('meow', 4)

Приложение

Код из lb3.py:

```
def LCS(str1, str2):
    m = len(str1)
    n = len(str2)
    lcs = [[0]*(n+1) for _ in range(1+m)]
    maxlen = 0
    endpos = 0

    # 0, i == 0 || j == 0
    # lcs[i-1][j-1] + 1, str1[i] == str[j]

    print(f"строка A: {str1}, длина = {m}, строки")
    print(f"строка B: {str2}, длина = {n}, столбцы")

    for i in range(1, m):
        lcs[i][0] = 0
    for j in range(1, n):
        lcs[0][j] = 0

    for i in range (1, m+1):
        for j in range (1, n+1):
            if str1[i-1] == str2[j-1]:
                lcs[i][j] = lcs[i-1][j-1] + 1
                if lcs[i][j] > maxlen:
                    maxlen = lcs[i][j]
                    endpos = i
                     print(f"\n\nновая подстрока: str1[{i-1}]='{str1[i-
1]}'    ==    str2[{j-1}]='{str2[j-1]}',\n    lcs[{i}][{j}]={lcs[i][j]},
maxlen={maxlen}, endpos={endpos}")
            else:
                lcs[i][j] = 0
        print(f"\nсостояние после i={i} (символ '{str1[i-1]}'):")
        for row in lcs[:i+1]:
            print(" ".join(f"{x:2d}" for x in row))
        print()

    if maxlen > 0:
        longsub = str1[endpos - maxlen : endpos]
        return longsub, maxlen
    else:
        return "Общих подстрок не существует", 0

str1 = "vmva"
str2 = "va"

print( LCS(str1, str2) )
```

Код из leven_equal.py:

```
rep, ins, dele = [int(x) for x in input().split()]

first = list(input().strip())
second = list(input().strip())
```

```
first_len = len(first)
second_len = len(second)

table = [ [0]*(second_len+1) for _ in range(first_len+1) ]

print(f"строка A: {first}, длина = {first_len}, строки")
print(f"строка B: {second}, длина = {second_len}, столбцы")
print(f"стоимости: replace={rep}, insert={ins}, delete={dele}\n")

for i in range (1, first_len+1): table[i][0] = i*dele
for j in range (1, second_len+1): table[0][j] = j*ins

print("начальная таблица (базовые стоимости):")
for row in table:
    print(" ".join(f"{x:3d}" for x in row))
print()

for i in range(1, 1+first_len):
    for j in range(1, 1+second_len):

        if first[i-1] == second[j-1]:
            m = 0
        else:
            m = rep

        table[i][j] = min(
            table[i - 1][j] + dele,
            table[i][j - 1] + ins,
            table[i - 1][j - 1] + m
        )

            print(f"после обработки строки i={i} (символ A[{i-
1}]='{first[i-1]}'):")
        for row in table:
            print(" ".join(f"{x:3d}" for x in row))
        print()

print(table[first_len][second_len])
```

Код из edit_script.py:

```
rep, ins, dele = [int(x) for x in input().split()]

first = list(input().strip())
second = list(input().strip())

first_len = len(first)
second_len = len(second)

print(f"строка A: {first}, длина = {first_len}, строки")
print(f"строка B: {second}, длина = {second_len}, столбцы")
print(f"стоимости: replace={rep}, insert={ins}, delete={dele}\n")

table = [ [0]*(second_len+1) for _ in range(first_len+1) ]

for i in range (1, first_len+1): table[i][0] = i*dele
```

```
    for j in range (1, second_len+1): table[0][j] = j*ins

    print("после базовой инициализации:")
    for row in table:
        print(" ".join(f"{x:3d}" for x in row))
    print()

    for i in range(1, 1+first_len):
        for j in range(1, 1+second_len):

            if first[i-1] == second[j-1]:
                m = 0
            else:
                m = rep

            table[i][j] = min(
                table[i - 1][j] + dele,
                table[i][j - 1] + ins,
                table[i - 1][j - 1] + m
            )
        print(f"после строки i={i} (A[{i-1}]='{first[i-1]}'):")
        for row in table:
            print(" ".join(f"{x:3d}" for x in row))
        print()

    print("редакционное предписание:")
    i, j = first_len, second_len

    prescripton = ""

    while i > 0 or j > 0 :
        print(f"текущая позиция (i={i}, j={j}), cost={table[i][j]}")

        if i > 0 and j > 0 and table[i][j] == table[i-1][j-1] + (0 if
first[i-1]==second[j-1] else rep):
            if first[i-1] == second[j-1]:
                prescripton += "M"
                 print(f"совпадение: A[{i-1}]='{first[i-1]}' == B[{j-
1}]='{second[j-1]}' -- M")
            else:
                prescripton += "R"
                    print(f"замена:  A[{i-1}]='{first[i-1]}'  --  B[{j-
1}]='{second[j-1]}' -- R")
            i -= 1
            j -= 1
        elif i > 0 and (table[i][j] == table[i - 1][j] + dele):
            prescripton += "D"
            i -= 1
            print(f"удаление: удаляем A[{i-1}]='{first[i-1]}' -- D")
        elif j > 0 and ( table[i][j] ==  table[i][j - 1] + ins):
            prescripton += "I"
            j -= 1
           print(f"вставка: вставляем B[{j-1}]='{second[j-1]}' -- I")

    print(prescripton[::-1])
    print("".join(first))
    print("".join(second))
```

Код из levenstein.py:

```
first = input().strip()
second = input().strip()
n, m = len(first), len(second)

print(f"строка A: {first}, длина = {n}, строки")
print(f"строка B: {second}, длина = {m}, столбцы")

prev = list(range(m + 1))  # dp[0][j] = j
print(f"prev[0..{m}] (строка 0): {prev}")

for i in range(1, n + 1):
    curr = [i] + [0] * m   # dp[i][0] = i
    print(f"\ni = {i} (символ A[{i-1}]='{first[i-1]}')")
    print(f"curr[0] = {curr[0]}")
    for j in range(1, m + 1):
        cost = 0 if first[i-1] == second[j-1] else 1
        curr[j] = min(
            prev[j] + 1,    # удаление
            curr[j-1] + 1,  # вставка
            prev[j-1] + cost # замена
        )
         print(f"  j={j} (B[{j-1}]='{second[j-1]}'), cost={cost},
curr[{j}] = {curr[j]}")
    print(f"prev: {prev}")
    print(f"curr: {curr}")
    prev = curr

print(prev[m])
```